

Real-Time Examples of Factory Method Design Pattern in C#

In this article, I will discuss the **Real-Time Examples of the Factory Method Design Pattern in C#**. Please read our previous article discussing the basic concepts of the [Factory Method Design Pattern in C#](#) with Examples. At the end of this article, you will understand the following Real-time Examples using the Factory Method Design Pattern in C#.

1. [Notification System](#)
2. [Document Format Converters](#)
3. [Report Generators](#)
4. [Payment Gateway Integration](#)
5. [Logistics Application](#)

Real-Time Example of Factory Method Design Pattern in C#: Notification System

Let's take a real-world example: Notification System. Suppose an application needs to send notifications to users, and the notification mode can be Email, SMS, or Push Notification. Using the Factory Method pattern, you can easily create a suitable notifier based on user preferences or conditions. Let us see how we can implement the above example using the Factory Method Design Pattern in C#:

```
using System;
namespace FactoryMethodDesignPattern
{
    //Product(Interface)
    public interface INotifier
    {
        void SendNotification(string message);
    }
    // Concrete Products
    public class EmailNotifier : INotifier
    {
        public void SendNotification(string message)
        {
            Console.WriteLine($"Sending Email: {message}");
            // Actual email sending logic goes here...
        }
    }
    public class SMSNotifier : INotifier
    {
        public void SendNotification(string message)
        {
            Console.WriteLine($"Sending SMS: {message}");
            // Actual SMS sending logic goes here...
        }
    }
    public class PushNotifier : INotifier
    {
        public void SendNotification(string message)
        {
            Console.WriteLine($"Sending Push Notification: {message}");
            // Actual push notification logic goes here...
        }
    }
    //Creator (Abstract Class)
```

```

public abstract class NotifierFactory
{
    public abstract INotifier CreateNotifier();
}
//Concrete Creators
public class EmailNotifierFactory : NotifierFactory
{
    public override INotifier CreateNotifier()
    {
        return new EmailNotifier();
    }
}
public class SMSNotifierFactory : NotifierFactory
{
    public override INotifier CreateNotifier()
    {
        return new SMSNotifier();
    }
}
public class PushNotifierFactory : NotifierFactory
{
    public override INotifier CreateNotifier()
    {
        return new PushNotifier();
    }
}
//Client Code
public class NotificationSystem
{
    public void NotifyUser(NotifierFactory factory, string message)
    {
        INotifier notifier = factory.CreateNotifier();
        notifier.SendNotification(message);
    }
}

// Testing the Factory Method Design Pattern
public class Program
{
    public static void Main()
    {
        var notificationSystem = new NotificationSystem();
        // For a user who prefers email notifications:
        notificationSystem.NotifyUser(new EmailNotifierFactory(), "This
is an email notification!");
        // For a user who prefers SMS notifications:
        notificationSystem.NotifyUser(new SMSNotifierFactory(), "This
is an SMS notification!");
        // For a user who prefers push notifications:
        notificationSystem.NotifyUser(new PushNotifierFactory(), "This
is a push notification!");
        Console.ReadKey();
    }
}

```

```
}
```

With this code in place, if you ever need to add another notification method (e.g., Webhook, Slack, etc.), you can easily extend the system by creating a new concrete notifier and its associated factory without modifying the existing code. When you run the above code, you will get the following output.

```
Sending Email: This is an email notification!
Sending SMS: This is an SMS notification!
Sending Push Notification: This is a push notification!
```

Real-Time Example of Factory Method Design Pattern in C#: Document Format Converters

Let's explore another real-world example: Document Format Converters. Imagine a document management system where users can export documents into formats such as PDF, DOCX, or TXT. Using the Factory Method pattern, you can instantiate the right converter based on the desired output format. Let us see how we can implement the above example using the Factory Method Design Pattern in C#:

```
using System;
namespace FactoryMethodDesignPattern
{
    //Product(Interface)
    public interface IDocumentConverter
    {
        void Convert(string inputFile, string outputFile);
    }
    // Concrete Products
    public class PdfConverter : IDocumentConverter
    {
        public void Convert(string inputFile, string outputFile)
        {
            Console.WriteLine($"Converting {inputFile} to PDF and saving as {outputFile}.");
            // Actual PDF conversion logic goes here...
        }
    }
    public class DocxConverter : IDocumentConverter
    {
        public void Convert(string inputFile, string outputFile)
        {
            Console.WriteLine($"Converting {inputFile} to DOCX and saving as {outputFile}.");
            // Actual DOCX conversion logic goes here...
        }
    }
    public class TxtConverter : IDocumentConverter
    {
        public void Convert(string inputFile, string outputFile)
        {
            Console.WriteLine($"Converting {inputFile} to TXT and saving as {outputFile}.");
            // Actual TXT conversion logic goes here...
        }
    }
}
```

```

}
//Creator (Abstract Class)
public abstract class DocumentConverterFactory
{
    public abstract IDocumentConverter CreateConverter();
}
//Concrete Creators
public class PdfConverterFactory : DocumentConverterFactory
{
    public override IDocumentConverter CreateConverter()
    {
        return new PdfConverter();
    }
}
public class DocxConverterFactory : DocumentConverterFactory
{
    public override IDocumentConverter CreateConverter()
    {
        return new DocxConverter();
    }
}
public class TxtConverterFactory : DocumentConverterFactory
{
    public override IDocumentConverter CreateConverter()
    {
        return new TxtConverter();
    }
}
//Client Code
public class DocumentService
{
    public void ExportDocument(DocumentConverterFactory factory, string
inputFile, string outputFile)
    {
        IDocumentConverter converter = factory.CreateConverter();
        converter.Convert(inputFile, outputFile);
    }
}

// Testing the Factory Method Design Pattern
public class Program
{
    public static void Main()
    {
        var documentService = new DocumentService();
        // User wants to export to PDF:
        documentService.ExportDocument(new PdfConverterFactory(),
"source.docx", "output.pdf");
        // User wants to export to DOCX:
        documentService.ExportDocument(new DocxConverterFactory(),
"source.pdf", "output.docx");
        // User wants to export to TXT:
        documentService.ExportDocument(new TxtConverterFactory(),
"source.pdf", "output.txt");
    }
}

```

```

        Console.ReadKey();
    }
}

```

With this approach, if you need to support another format, such as ODT or RTF, you can effortlessly introduce the new converter and its associated factory without changing the existing code. When you run the above code, you will get the following output.

```

Converting source.docx to PDF and saving as output.pdf.
Converting source.pdf to DOCX and saving as output.docx.
Converting source.pdf to TXT and saving as output.txt.

```

Real-Time Example of Factory Method Design Pattern in C#: Report Generators

Let's explore yet another example: Report Generators. Let's assume we have an analytics application allowing users to generate reports in different formats: Chart, Tabular, or Summary. Using the Factory Method pattern, we can instantiate the appropriate report generator based on the user's selection. Let us see how we can implement the above example using the Factory Method Design Pattern in C#:

```

using System;
namespace FactoryMethodDesignPattern
{
    //Product(Interface)
    public interface IReportGenerator
    {
        void GenerateReport(string data);
    }
    // Concrete Products
    public class ChartReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating Chart Report with data:
{data}");
            // Actual logic to generate a chart report...
        }
    }
    public class TabularReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating Tabular Report with data:
{data}");
            // Actual logic to generate a tabular report...
        }
    }
    public class SummaryReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating Summary Report with data:
{data}");
            // Actual logic to generate a summary report...
        }
    }
}

```

```

    }
}
//Creator (Abstract Class)
public abstract class ReportGeneratorFactory
{
    public abstract IReportGenerator CreateReportGenerator();
}
//Concrete Creators
public class ChartReportGeneratorFactory : ReportGeneratorFactory
{
    public override IReportGenerator CreateReportGenerator()
    {
        return new ChartReportGenerator();
    }
}
public class TabularReportGeneratorFactory : ReportGeneratorFactory
{
    public override IReportGenerator CreateReportGenerator()
    {
        return new TabularReportGenerator();
    }
}
public class SummaryReportGeneratorFactory : ReportGeneratorFactory
{
    public override IReportGenerator CreateReportGenerator()
    {
        return new SummaryReportGenerator();
    }
}
//Client Code
public class AnalyticsService
{
    public void CreateReport(ReportGeneratorFactory factory, string
data)
    {
        IReportGenerator reportGenerator =
factory.CreateReportGenerator();
        reportGenerator.GenerateReport(data);
    }
}

// Testing the Factory Method Design Pattern
public class Program
{
    public static void Main()
    {
        var analyticsService = new AnalyticsService();
        // User wants a Chart report:
        analyticsService.CreateReport(new
ChartReportGeneratorFactory(), "ChartData123");
        // User wants a Tabular report:
        analyticsService.CreateReport(new
TabularReportGeneratorFactory(), "TabularData456");
        // User wants a Summary report:

```

```

        analyticsService.CreateReport(new
SummaryReportGeneratorFactory(), "SummaryData789");
        Console.ReadKey();
    }
}
}

```

This structure provides flexibility. In the future, if you need to support other report types, you can seamlessly add new report generators and their factories without altering the core logic of your application. When you run the above code, you will get the following output.

```

Generating Chart Report with data: ChartData123
Generating Tabular Report with data: TabularData456
Generating Summary Report with data: SummaryData789

```

Real-Time Example of Factory Method Design Pattern in C#: Payment Gateway Integration

Assume you're building an e-commerce application and want to support multiple payment methods, like Credit Card, PayPal, and Bitcoin. Using the Factory Method pattern, you can instantiate the appropriate payment gateway based on the user's choice. Let us see how we can implement the above example using the Factory Method Design Pattern in C#:

```

using System;
namespace FactoryMethodDesignPattern
{
    //Product(Interface)
    public interface IPaymentGateway
    {
        void ProcessPayment(decimal amount);
    }
    // Concrete Products
    public class CreditCardPaymentGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing credit card payment of amount:
${amount}");
            // Actual logic to process credit card payment...
        }
    }
    public class PayPalPaymentGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing PayPal payment of amount:
${amount}");
            // Actual logic to process PayPal payment...
        }
    }
    public class BitcoinPaymentGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
        {

```

```

        Console.WriteLine($"Processing Bitcoin payment of amount:
${amount}");
        // Actual logic to process Bitcoin payment...
    }
}
//Creator (Abstract Class)
public abstract class PaymentGatewayFactory
{
    public abstract IPaymentGateway CreatePaymentGateway();
}
//Concrete Creators
public class CreditCardPaymentGatewayFactory : PaymentGatewayFactory
{
    public override IPaymentGateway CreatePaymentGateway()
    {
        return new CreditCardPaymentGateway();
    }
}
public class PayPalPaymentGatewayFactory : PaymentGatewayFactory
{
    public override IPaymentGateway CreatePaymentGateway()
    {
        return new PayPalPaymentGateway();
    }
}
public class BitcoinPaymentGatewayFactory : PaymentGatewayFactory
{
    public override IPaymentGateway CreatePaymentGateway()
    {
        return new BitcoinPaymentGateway();
    }
}
//Client Code
public class ECommercePlatform
{
    public void Checkout(PaymentGatewayFactory factory, decimal amount)
    {
        IPaymentGateway paymentGateway =
factory.CreatePaymentGateway();
        paymentGateway.ProcessPayment(amount);
    }
}

// Testing the Factory Method Design Pattern
public class Program
{
    public static void Main()
    {
        var platform = new ECommercePlatform();
        // User selects Credit Card as the payment method:
        platform.Checkout(new CreditCardPaymentGatewayFactory(),
100.50M);
        // User selects PayPal as the payment method:
        platform.Checkout(new PayPalPaymentGatewayFactory(), 150.75M);
    }
}

```

```

        // User selects Bitcoin as the payment method:
        platform.Checkout(new BitcoinPaymentGatewayFactory(), 50.30M);
        Console.ReadKey();
    }
}
}

```

When you run the program based on the factory instantiated, it will use the Factory Method to create an instance of the appropriate payment method and process the payment. This approach provides flexibility to easily add more payment methods like Bank Transfer, Crypto, etc., in the future without altering the client code that uses these payment methods. When you run the above code, you will get the following output.

```

Processing credit card payment of amount: $100.50
Processing PayPal payment of amount: $150.75
Processing Bitcoin payment of amount: $50.30

```

Real-Time Example of Factory Method Design Pattern in C#: Logistics Application

Assume we are building a logistics application where different modes of transport deliver goods. We want to calculate the delivery cost based on the transport mode (for simplicity, let's consider Truck and Ship as transport modes). We will use the Factory Method pattern to create transport instances and calculate the delivery cost. Let us see how we can implement the above example using the Factory Method Design Pattern in C#:

```

using System;
namespace FactoryMethodDesignPattern
{
    //Product(Interface)
    public interface ITransport
    {
        double GetDeliveryCost(int distance);
    }
    // Concrete Products
    public class Truck : ITransport
    {
        public double GetDeliveryCost(int distance)
        {
            // Cost per mile for a truck might be $1.00
            return 1.00 * distance;
        }
    }
    public class Ship : ITransport
    {
        public double GetDeliveryCost(int distance)
        {
            // Cost per mile for a ship might be $0.50
            return 0.50 * distance;
        }
    }
    //Creator (Abstract Class)
    public abstract class TransportFactory
    {
        public abstract ITransport CreateTransport();
    }
    //Concrete Creators

```

```

public class TruckFactory : TransportFactory
{
    public override ITransport CreateTransport()
    {
        return new Truck();
    }
}
public class ShipFactory : TransportFactory
{
    public override ITransport CreateTransport()
    {
        return new Ship();
    }
}

// Testing the Factory Method Design Pattern
public class Program
{
    public static void Main()
    {
        TransportFactory factory;
        ITransport transport;
        factory = new TruckFactory();
        transport = factory.CreateTransport();
        Console.WriteLine($"Truck delivery cost:
${transport.GetDeliveryCost(100)} for 100 miles.");
        factory = new ShipFactory();
        transport = factory.CreateTransport();
        Console.WriteLine($"Ship delivery cost:
${transport.GetDeliveryCost(100)} for 100 miles.");
        Console.ReadKey();
    }
}

```

When you run the program, it will use the Factory Method to create instances of Truck and Ship and then calculate the delivery cost for each transport mode over 100 miles. This design lets you add more modes of transport in the future without altering the client code that uses the transport objects. When you run the above code, you will get the following output.

```

Truck delivery cost: $100 for 100 miles.
Ship delivery cost: $50 for 100 miles.

```

Advantages and Disadvantages of Factory Method Design Pattern in C#

The Factory Method Design Pattern is widely used in object-oriented programming, and like all design patterns, it has pros and cons.

Advantages of the Factory Method Design Pattern:

- **Loose Coupling:** The pattern promotes loose coupling by separating object creation from its usage. This means the client code does not need to know the specific class that needs to be instantiated.

- **Single Responsibility Principle:** It adheres to the Single Responsibility Principle by separating the creation and business logic, making the system easier to manage and extend.
- **Open/Closed Principle:** The pattern supports the Open/Closed Principle. It's easy to introduce new types of products without changing the existing client code.
- **Flexibility and Extensibility:** It provides more flexibility in code since it delegates the instantiation logic to subclasses. This makes the system more adaptable and extendable.
- **Ease of Testing:** The pattern simplifies unit testing as it allows for the use of mock objects or stubs in place of actual implementations.

Disadvantages of the Factory Method Design Pattern:

- **Complexity:** The pattern can add unnecessary complexity to the code if the number of creator classes increases, especially in simple scenarios where such a level of abstraction is unnecessary.
- **Requires Proper Understanding:** Misuse or overuse of the Factory Method can lead to a design that's harder to understand and maintain. It requires a good understanding of the pattern and its appropriate use cases.
- **Subclass Proliferation:** In some cases, creating a subclass for each new product type can lead to a proliferation of subclasses, making the system harder to understand and maintain.
- **Indirect Code Flow:** The indirect nature of object creation can sometimes make the code flow harder to follow, which might impact code readability.
- **Dependency on Subclasses:** Since object creation is delegated to subclasses, there's a dependency on these subclasses. Changes in subclasses might impact the object creation process.

In the next article, I will discuss the **Abstract Factory Design Pattern in C#** with Examples. Here, in this article, I try to explain **Real-Time Examples of Factory Method Design Patterns in C#**. I hope you enjoy this Factory Method Design Pattern Real-Time Examples using C# article.